CSE 493L Section 2

Parallel Prefix Sum

Check In !

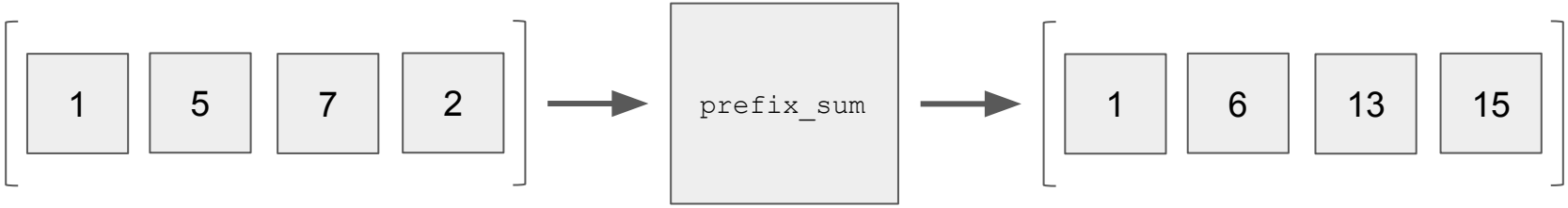
- What was most difficult about HW1?
- Any feedback for how to improve it?
- Any lingering questions?

Homework 2: Parallel Prefix Sum - Due Dates

- **No test case to submit for this homework!**
- **Wednesday, January 21**: code, questions - with 2 day grace period

Prefix Sum

- `prefix_sum(A: [T]) -> [T]` computes the running totals of an input sequence.



- “cumulative sum”, “scan”, etc…

Prefix Sum

- `prefix_sum(A: [T]) -> [T]` computes the running totals of an input sequence.

- “cumulative sum”, “scan”, etc…

Prefix Sum

- `prefix_sum(A: [T]) -> [T]` computes the running totals of an input sequence.

- “cumulative sum”, “scan”, etc…

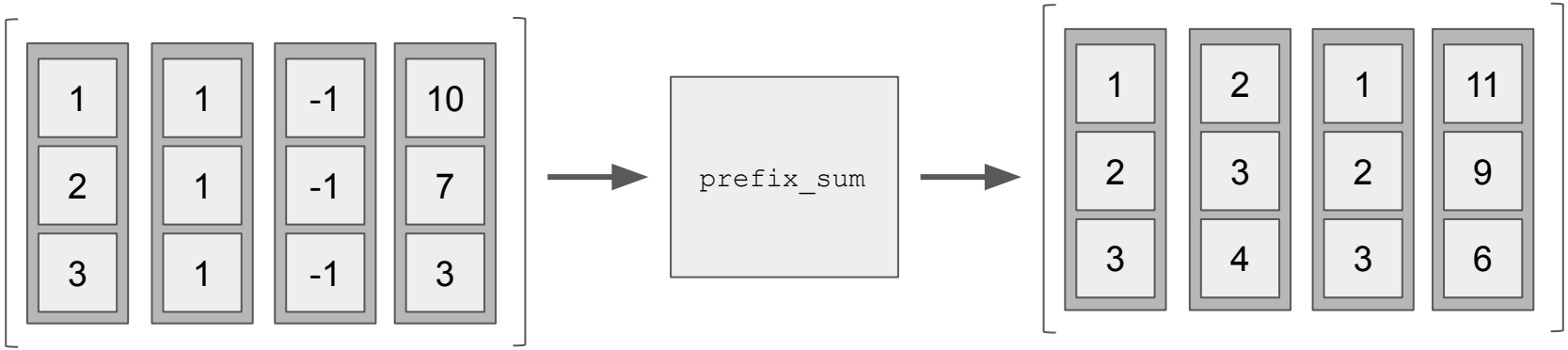
Prefix Sum

- `prefix_sum(A: [T]) -> [T]` computes the running totals of an input sequence.

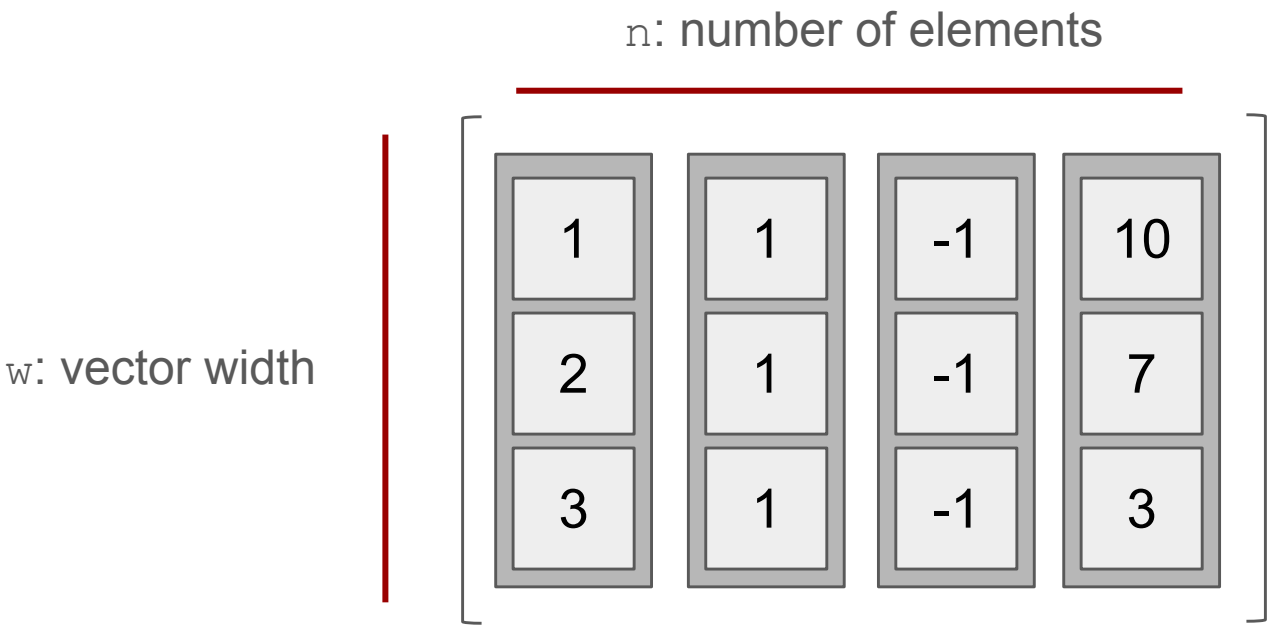
- “cumulative sum”, “scan”, etc…

Prefix Sum

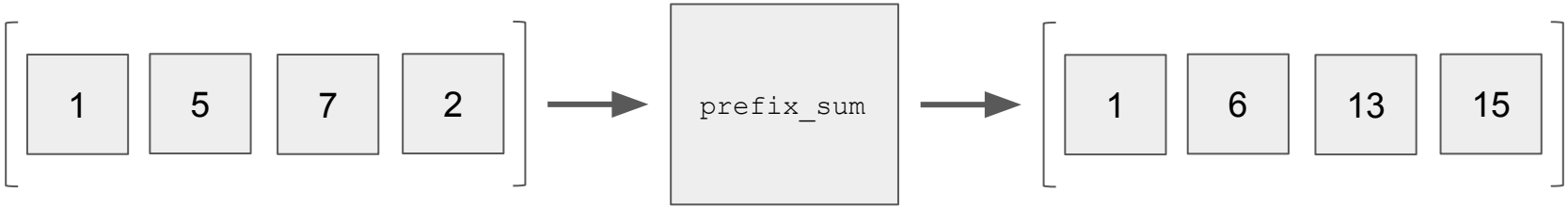
- Can be used beyond integer addition!
  - Works with any associative binary operation over any `T`
  - In HW2 we add vectors of floats!



Prefix Sum



Prefix Sum



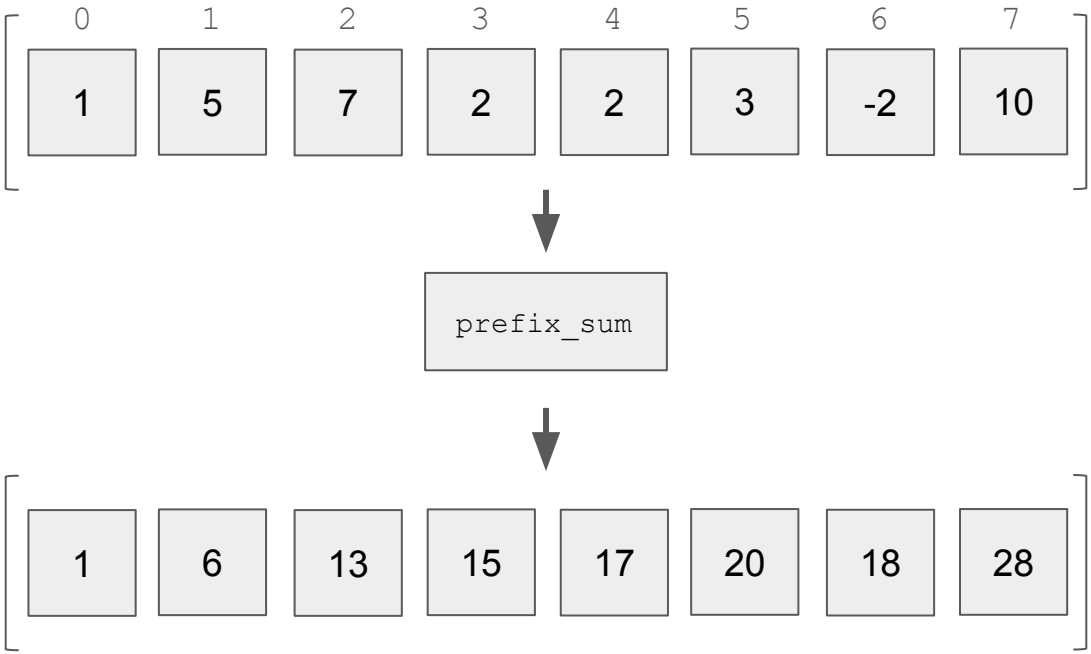
$$\texttt{output[n]} = \sum_{i=0}^{n} \texttt{input[i]}$$

or

```
output[0] = input[0]
output[n] = output[n-1] + input[n]
```

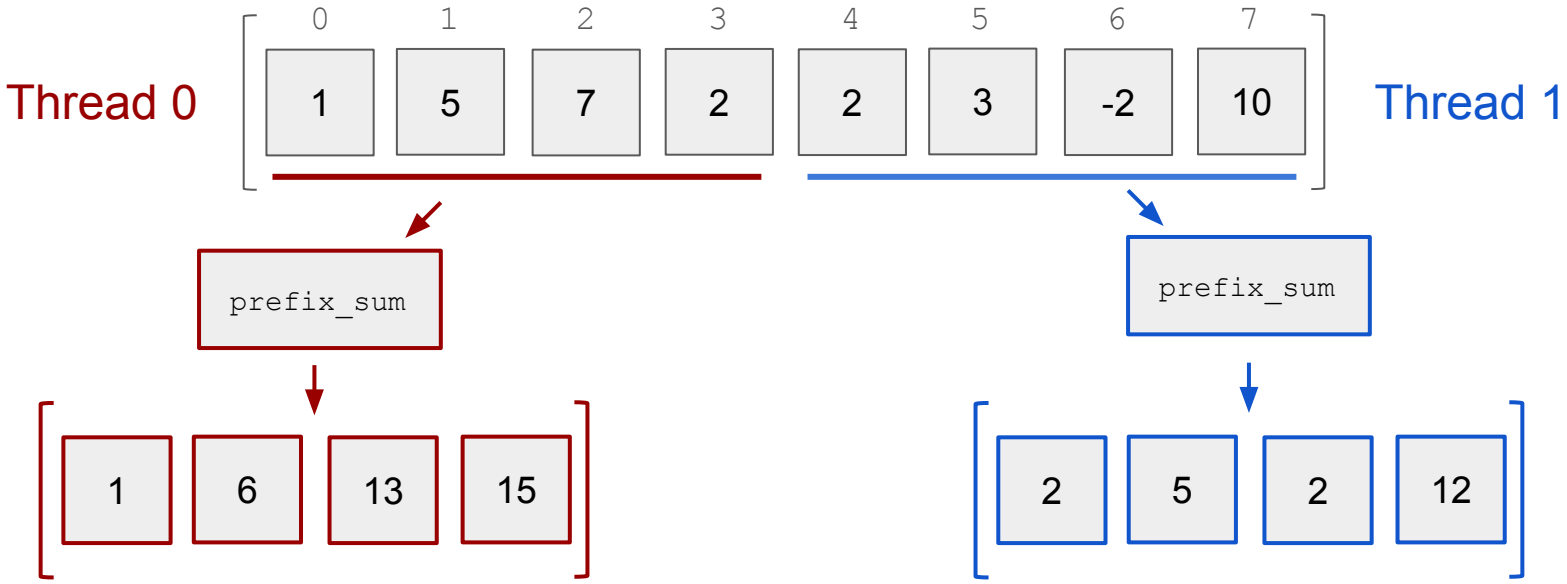
Parallel Prefix Sum

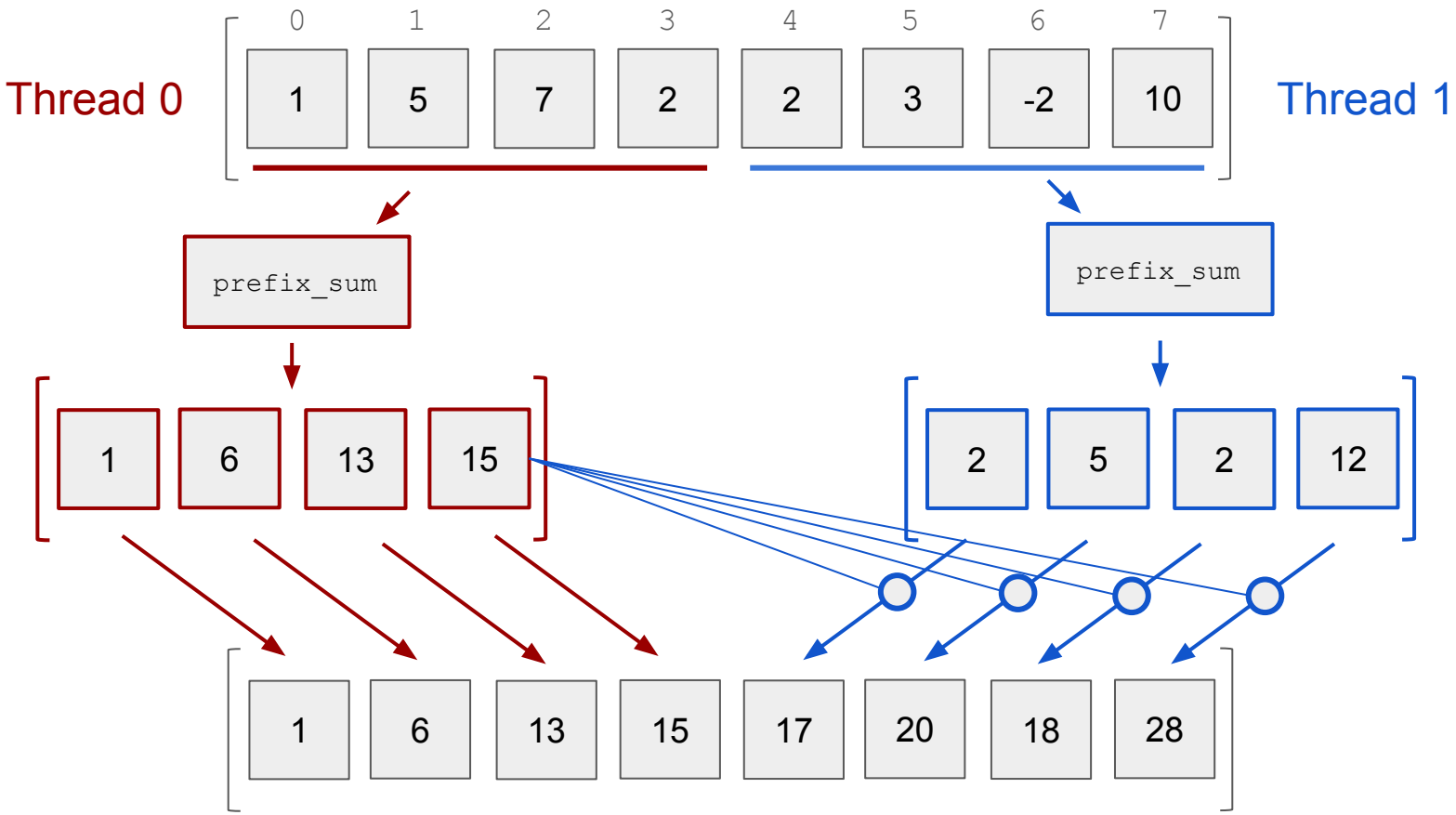
- How could we decompose this problem to expose parallelism?



Parallel Prefix Sum

- How could we decompose this problem to expose parallelism?

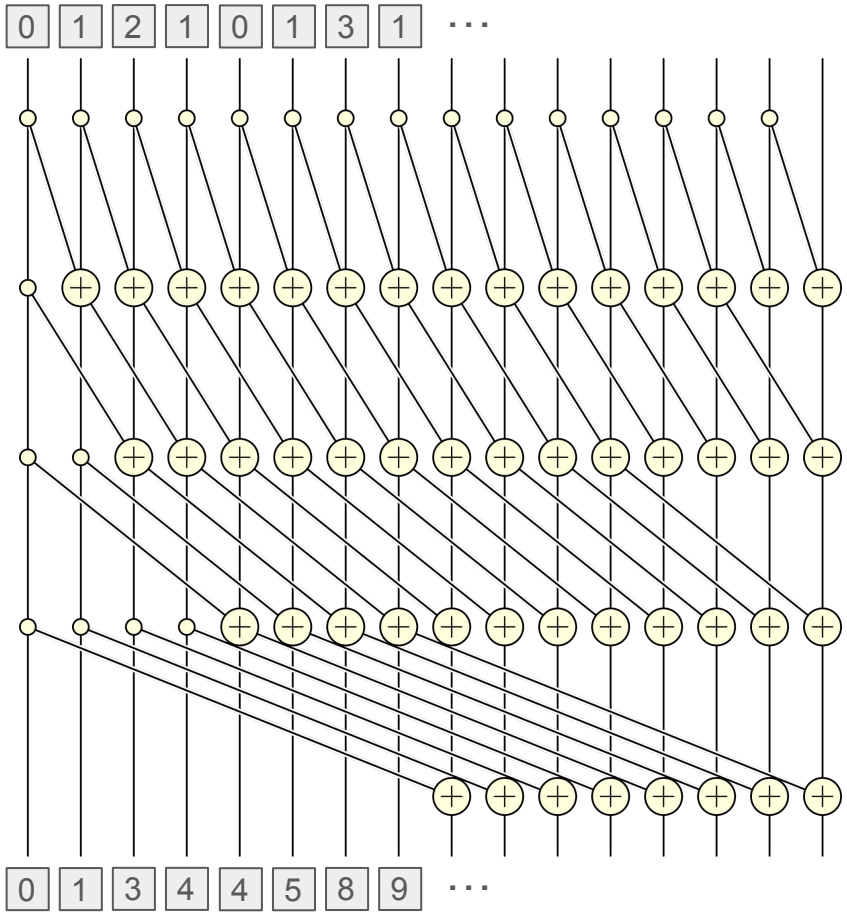



Thread 0
Thread 1
0
1
2
3
4
5
6
7
1
5
7
2
2
3
-2
10
prefix_sum
prefix_sum
1
6
13
15
2
5
2
12
1
6
13
15
17
20
18
28

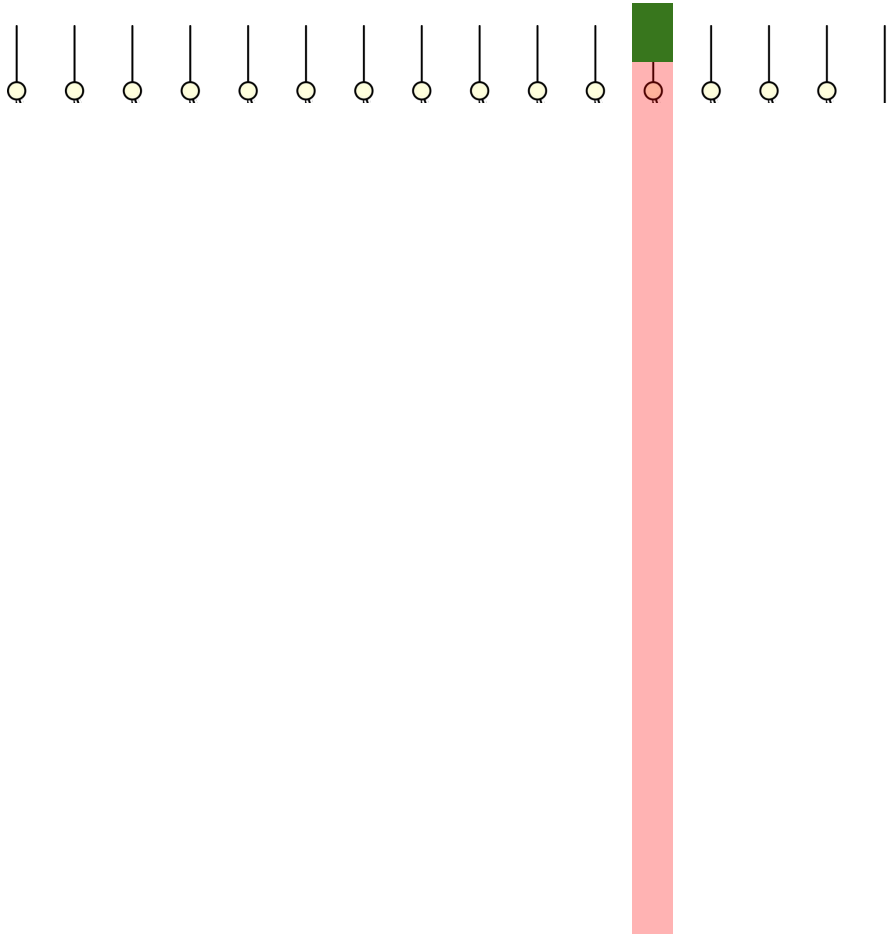
Highly-Parallel Prefix Sum

“Circuit diagram” representation.

- Top represents the input array.
- Bottom represents the output array.
- Each element is a line.



Highly-Parallel Prefix Sum



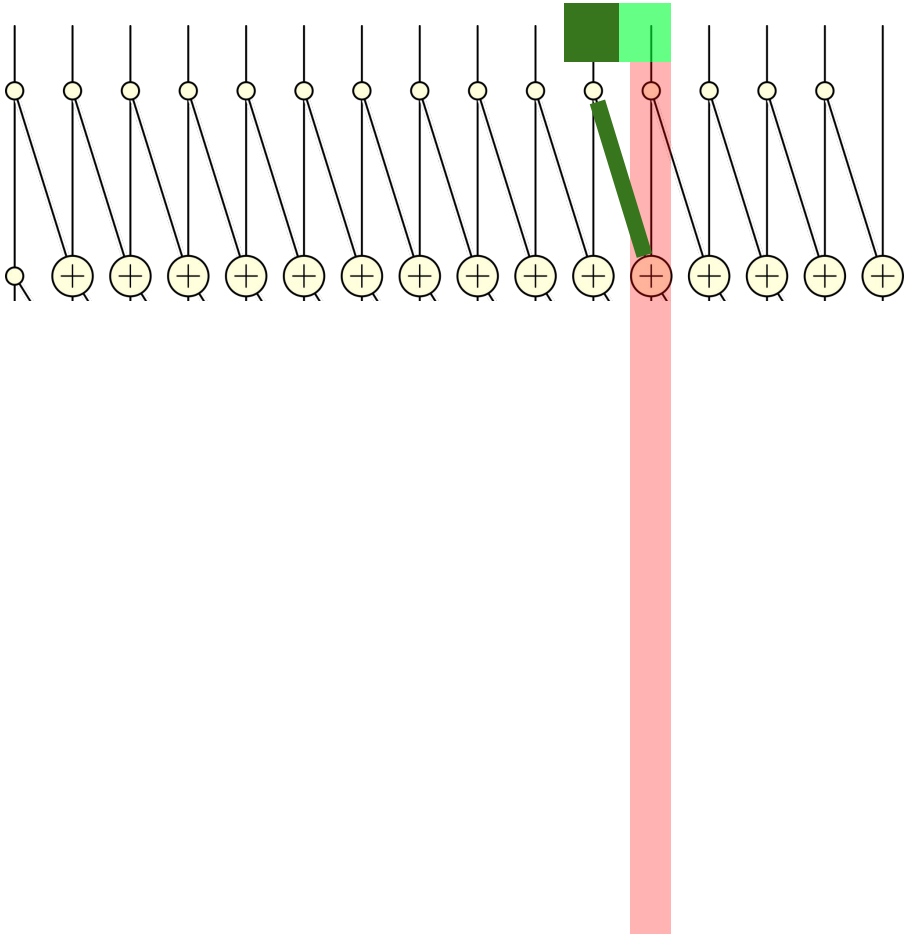
Red element holds the sum of

1 input element

Highly-Parallel Prefix Sum

Red element holds the sum of

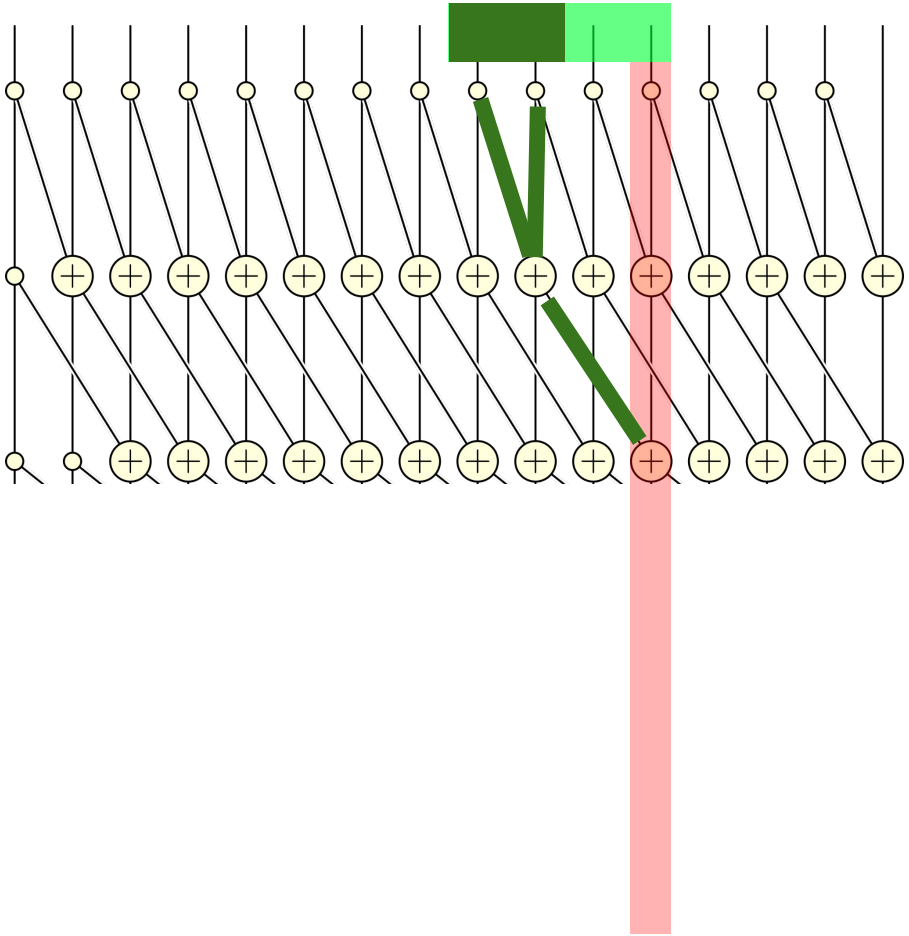
2 input elements



Highly-Parallel Prefix Sum

Red element holds the sum of

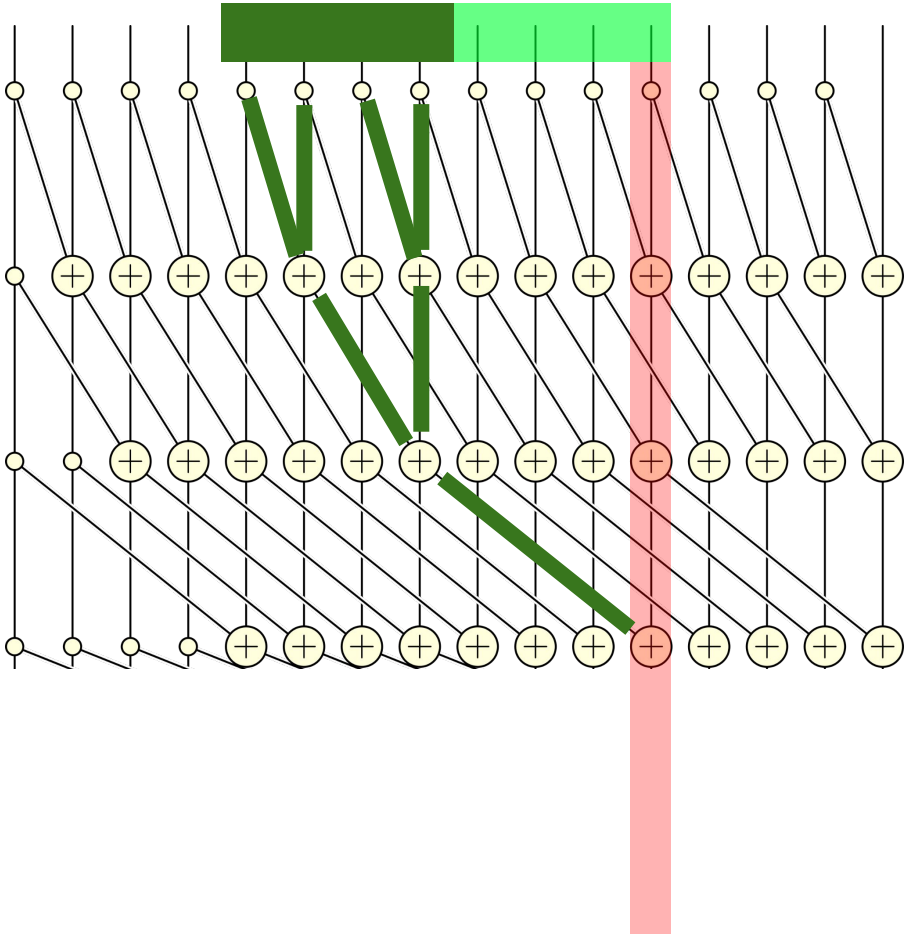
4 input elements



Highly-Parallel Prefix Sum

Red element holds the sum of

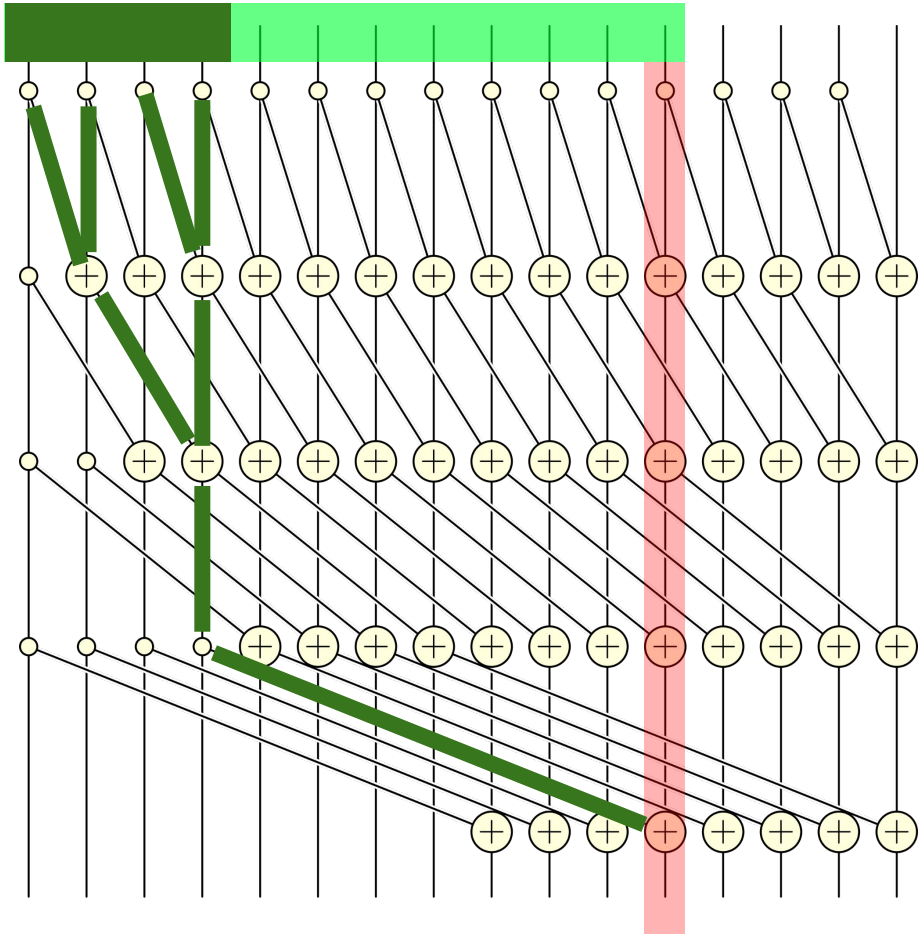
8 input elements



Highly-Parallel Prefix Sum

Red element holds the sum of

12 input elements

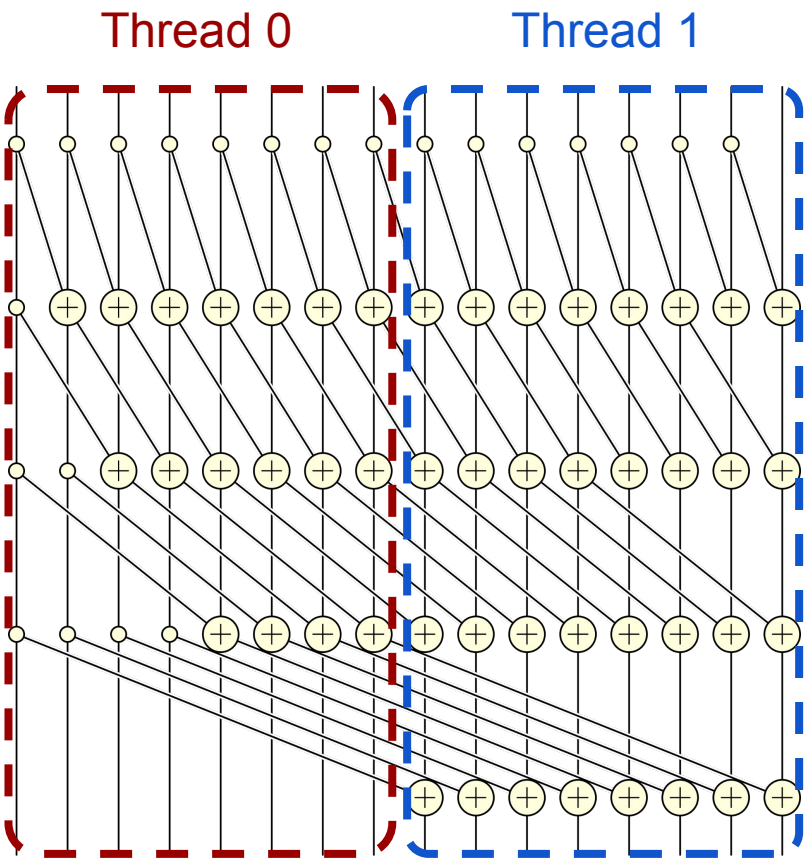


Where are the threads?

Each thread (with `tid`) is responsible for range:

```
[ tid*n/t, (tid+1)*n/t )
```

- Valid, mutually exclusive ranges.
- Each thread must be able to handle any range (non-power-of-two, etc.)
- If `t > n`, some threads get empty ranges like `[5,5)`. These simply don't do any work.

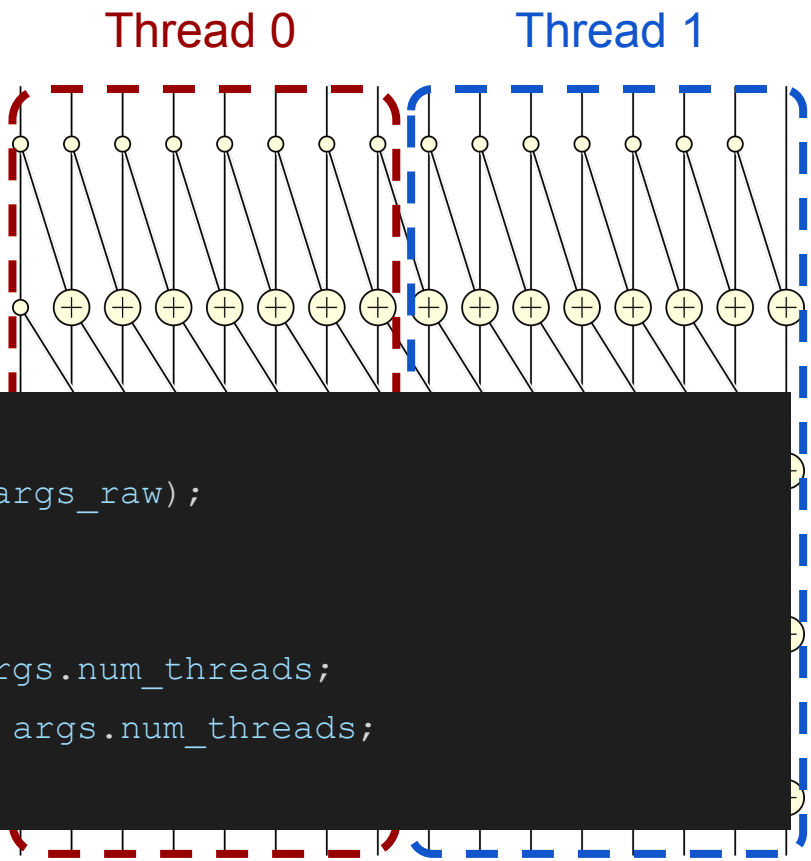


Where are the threads?

Each thread (with `tid`) is responsible for range:

`[ tid*n/t, (tid+1)*n/t )`

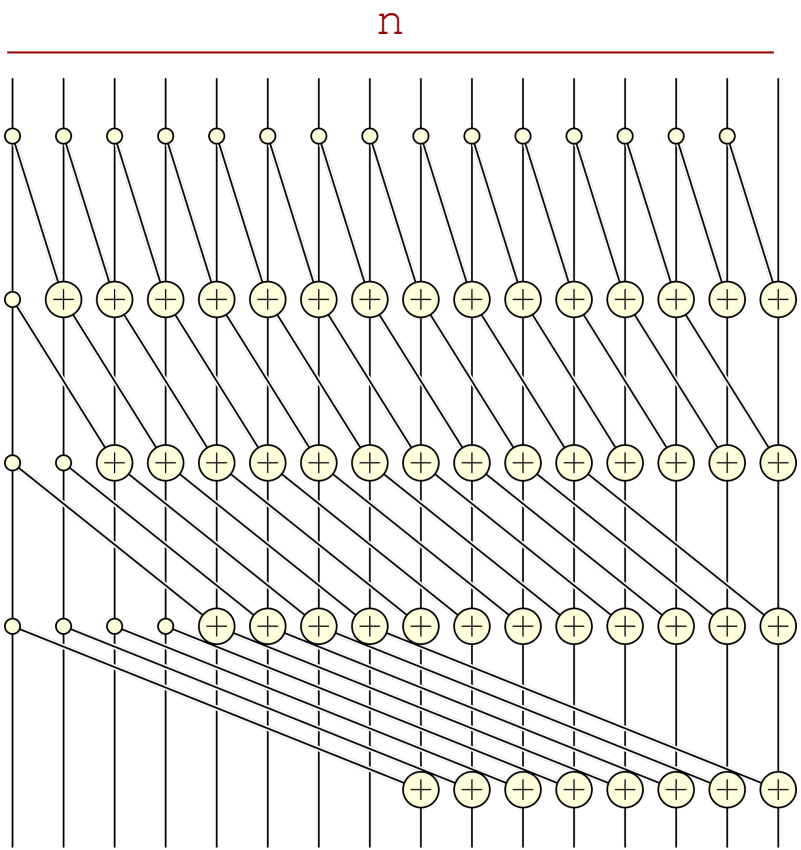
- Valid, mutually exclusive ranges.




```
void* worker(void *args_raw) {
    ThreadArg args = *static_cast<ThreadArg*>(args_raw);
    size_t input_size = args.input.size();
    // YOUR CODE HERE
    size_t start_i = args.tid * input_size / args.num_threads;
    size_t end_i = (args.tid+1) * input_size / args.num_threads;
}
```

Highly-Parallel Prefix Sum

```
for i in 0..log2(n) do
    for j in 0..n do in parallel
        if j-2^i ≥ 0 then
            x_{i+1}[j] ← x_i[j] + x_i[j-2^i]
        else
            x_{i+1}[j] ← x_i[j]
```

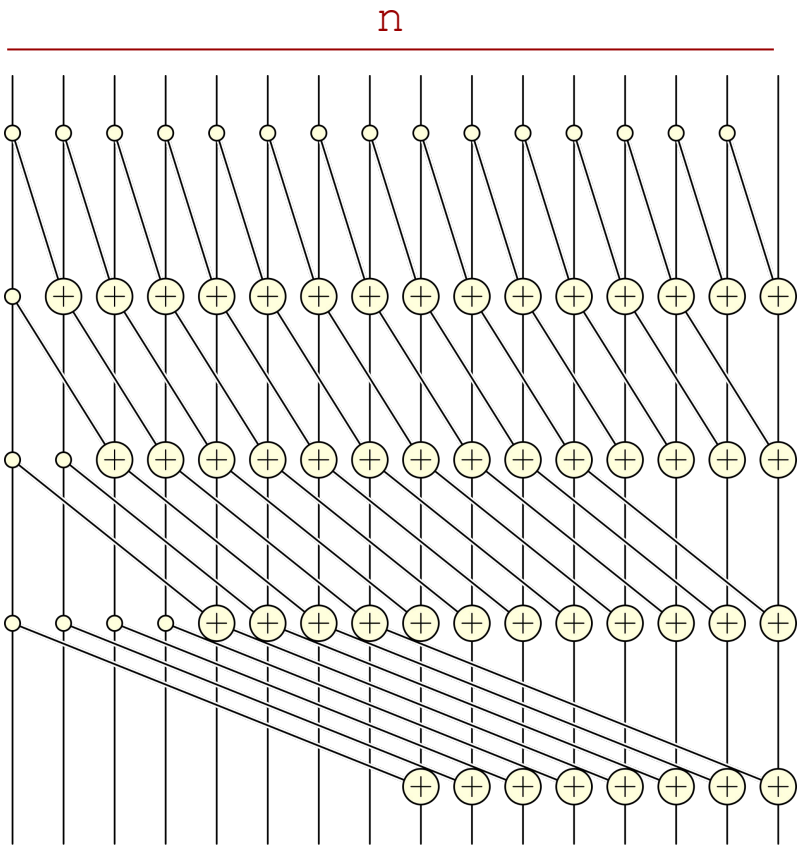


Highly-Parallel Prefix Sum

Where is synchronization required?

Runtime if there is **1** thread?

Runtime if there are **n** threads?



Highly-Parallel Prefix Sum

Where is synchronization required?

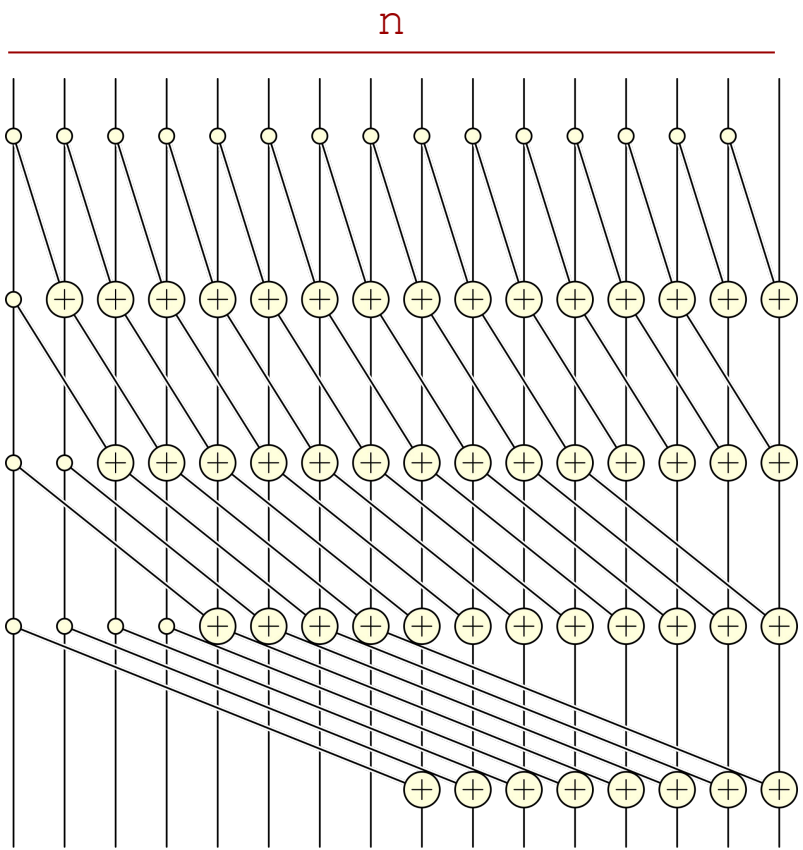
Between every step, but can optimize

Runtime if there is **1** thread?

$O(\ n \log_2 n\ )$ **"NOT work efficient"!**

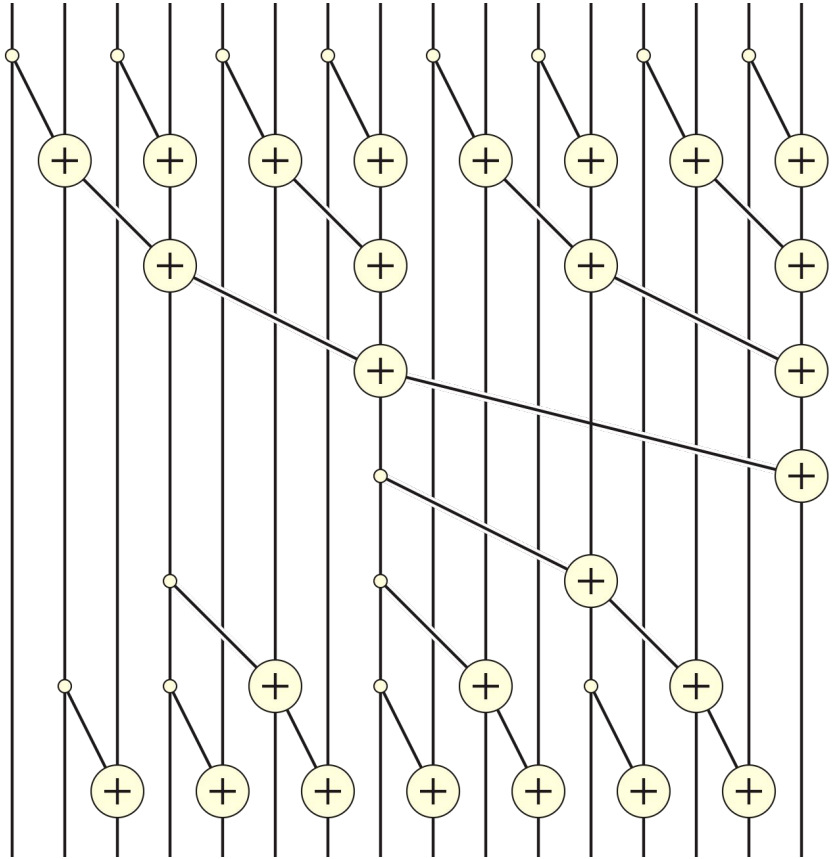
Runtime if there are **n** threads?

$O(\ \log_2 n\ )$



Work-Efficient Parallel Prefix Sum

This is what you'll implement in HW2.



Some useful references:

en.wikipedia.org/wiki/Prefix_sum#Parallel_algorithms

https://youtu.be/OO3o14cINbo?si=LpLBcyHsSi5X7Jvi

Work-Efficient Parallel Prefix Sum

This is what you'll implement in HW2.

It's easiest to understand this algorithm via a recursive definition, so let's assume that we have a function `prefix_sum`.

We will find an algorithm for `prefix_sum` for increasing length arrays.

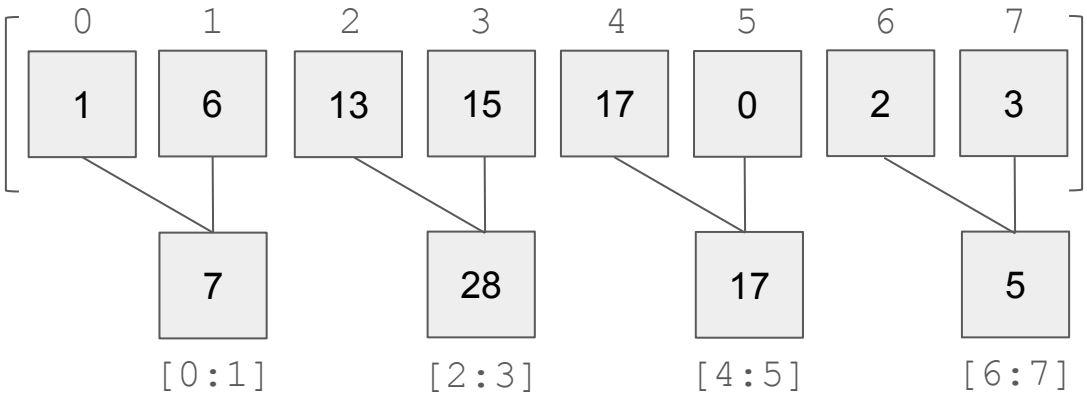
In the base case, let `x` be a single element array, intuitively,

```
prefix_sum(x)= x
```

Work-Efficient Parallel Prefix Sum

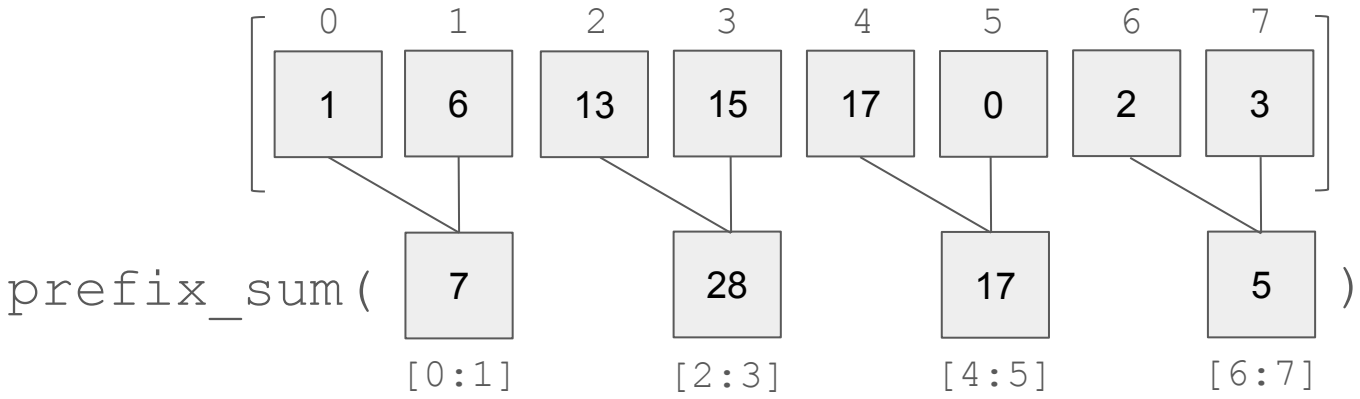
0	1	2	3	4	5	6	7
1	6	13	15	17	0	2	3

Work-Efficient Parallel Prefix Sum



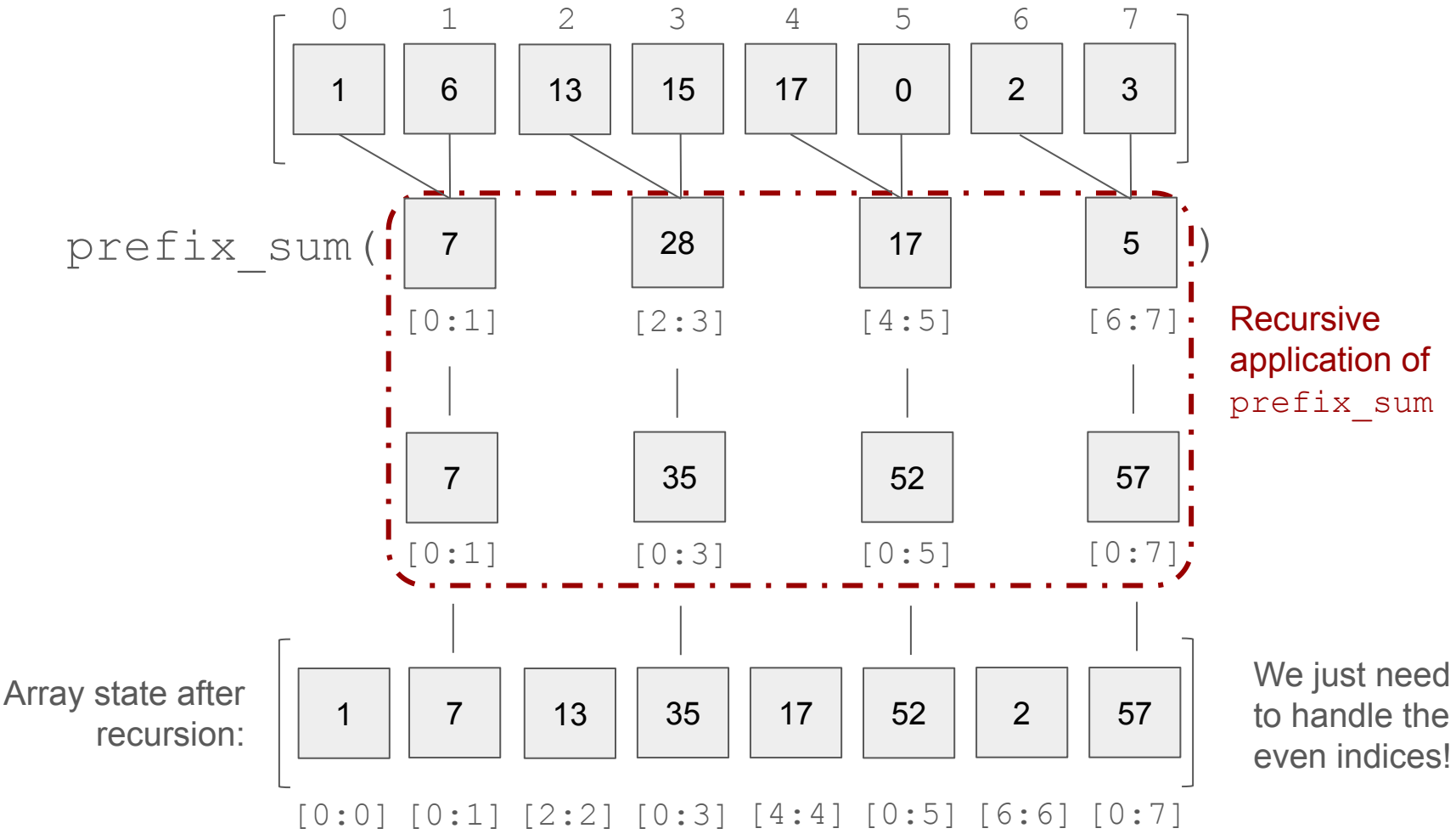
`[a:b]` → Sum of elements from `a` to `b`, inclusive.

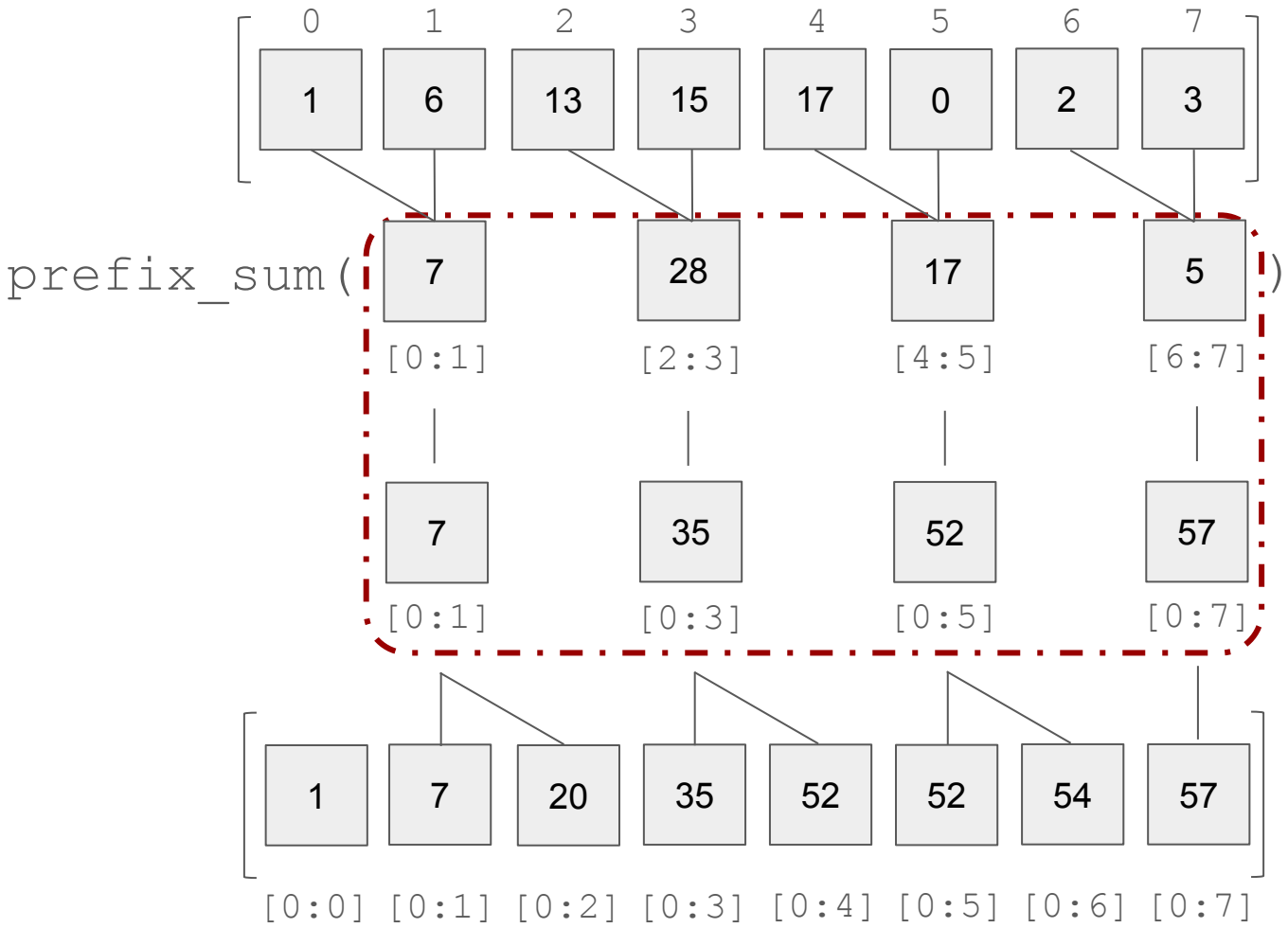
Work-Efficient Parallel Prefix Sum



What happens if we take the prefix sum* of the intermediate values?

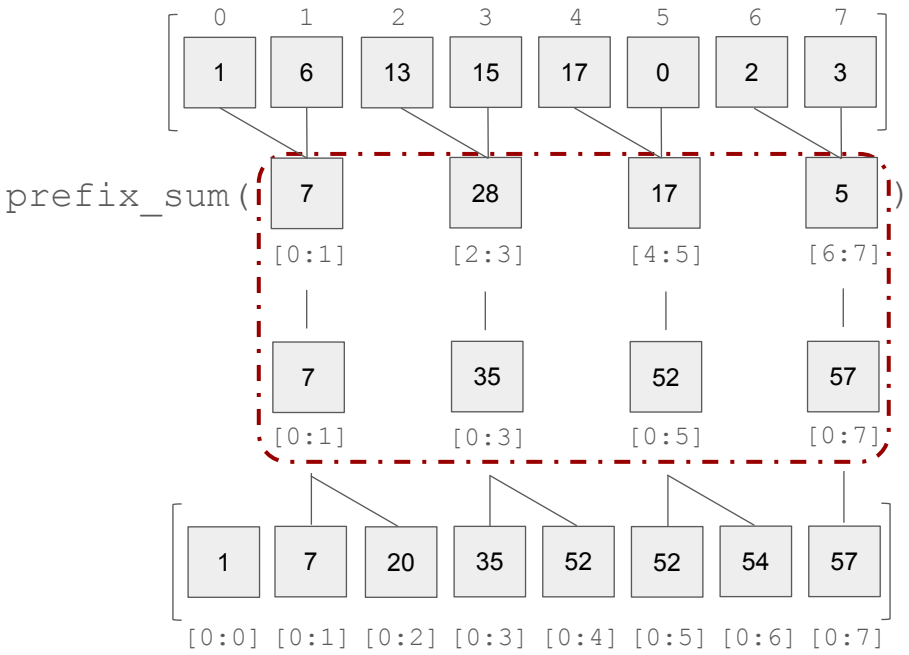
*We can do this as the size of the array is now smaller!

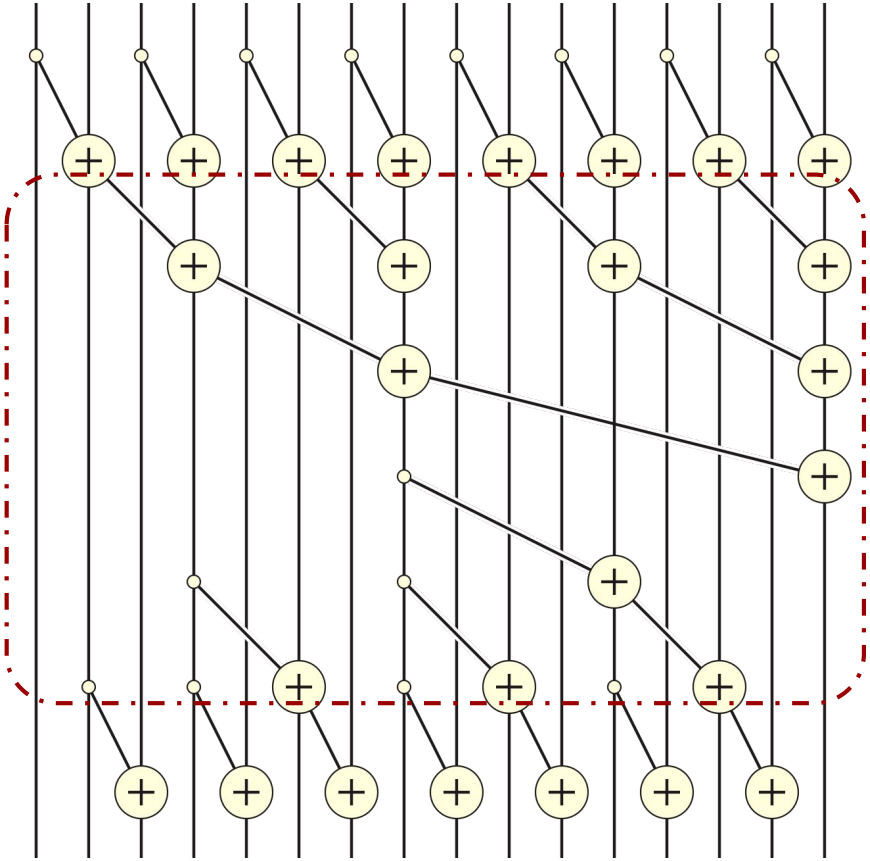

0 1 2 3 4 5 6 7
1 6 13 15 17 0 2 3
prefix_sum(
7 28 17 5
[0:1] [2:3] [4:5] [6:7]
7 35 52 57
[0:1] [0:3] [0:5] [0:7]
)
Recursive application of prefix_sum
Array state after recursion:
1 7 13 35 17 52 2 57
[0:0] [0:1] [2:2] [0:3] [4:4] [0:5] [6:6] [0:7]
We just need to handle the even indices!



And we're done!

These two diagrams are equivalent, the right is just "unwrapping" the recursive call made on the left.



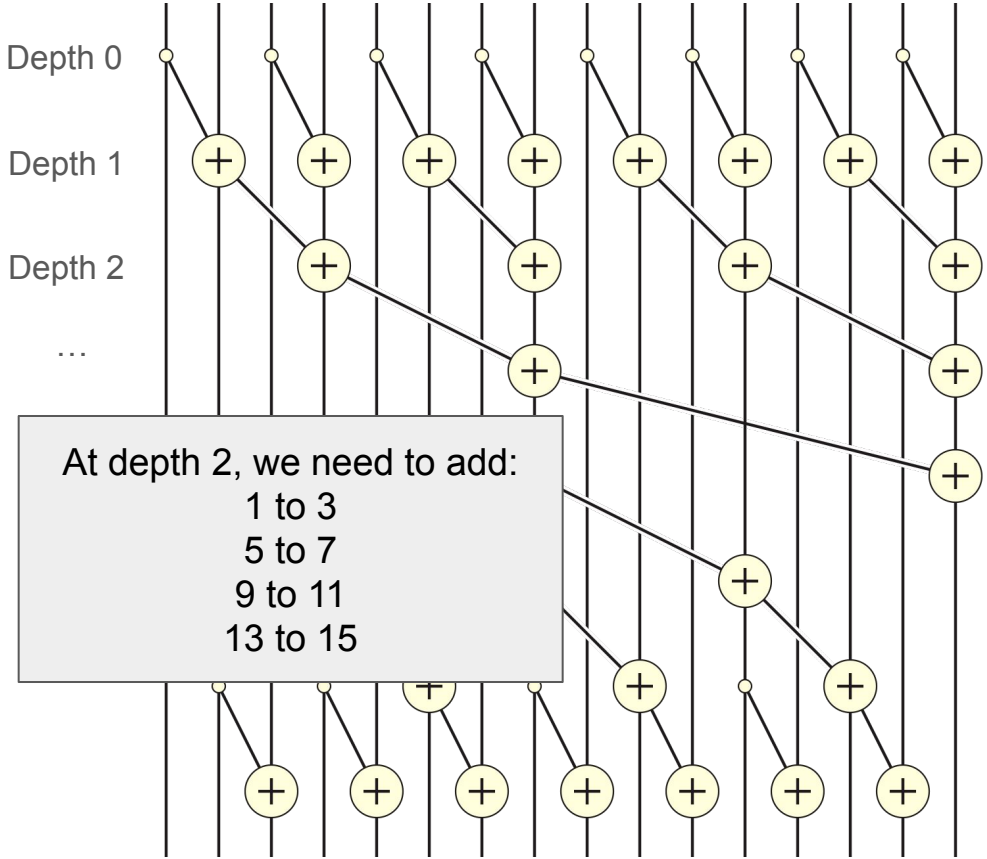


Work-Efficient Parallel Prefix Sum

We want an iterative algorithm

- More optimized for GPUs (with their small amount of stack memory)

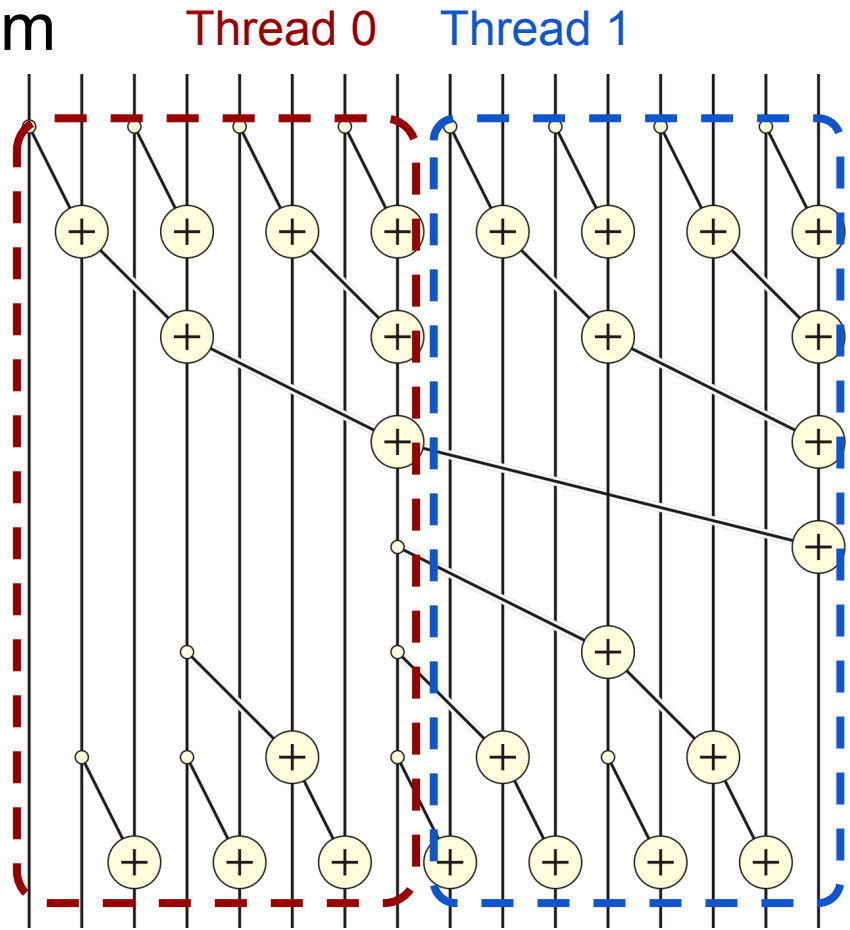
At every depth, use some power-of-two math to determine which indices need to be added together.



Work-Efficient Parallel Prefix Sum

Very similar parallelization strategy!

```
[ tid*n/t, (tid+1)*n/t )
```

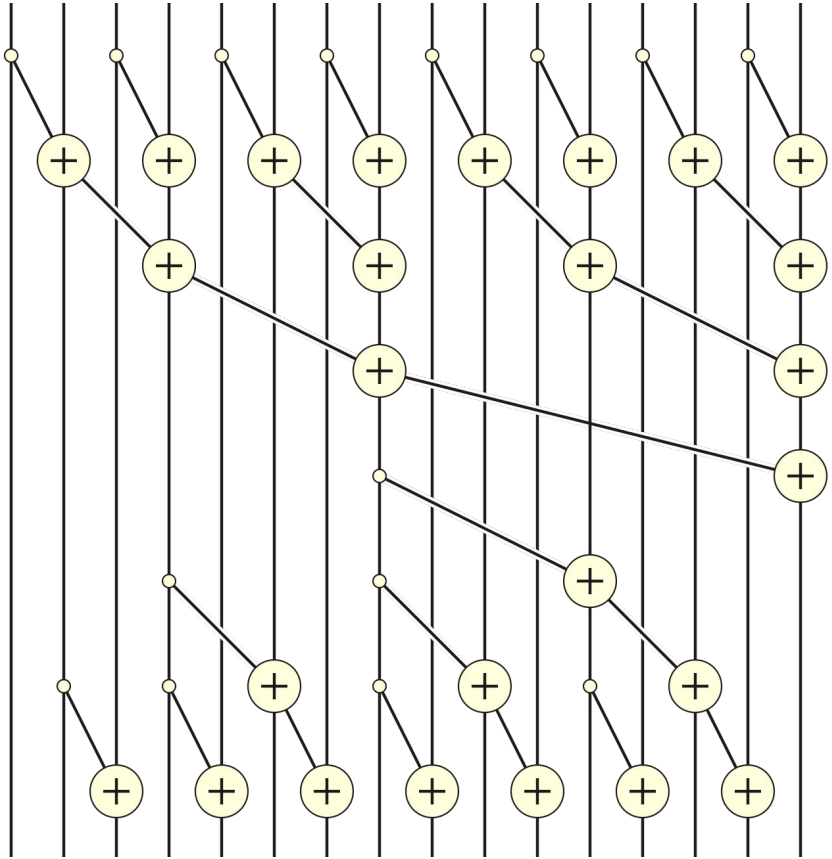


Work-Efficient Parallel Prefix Sum

Where is synchronization required?

Runtime if there is **1** thread?

Runtime if there are **n** threads?



Work-Efficient Parallel Prefix Sum

Where is synchronization required?

Between every step, but can optimize

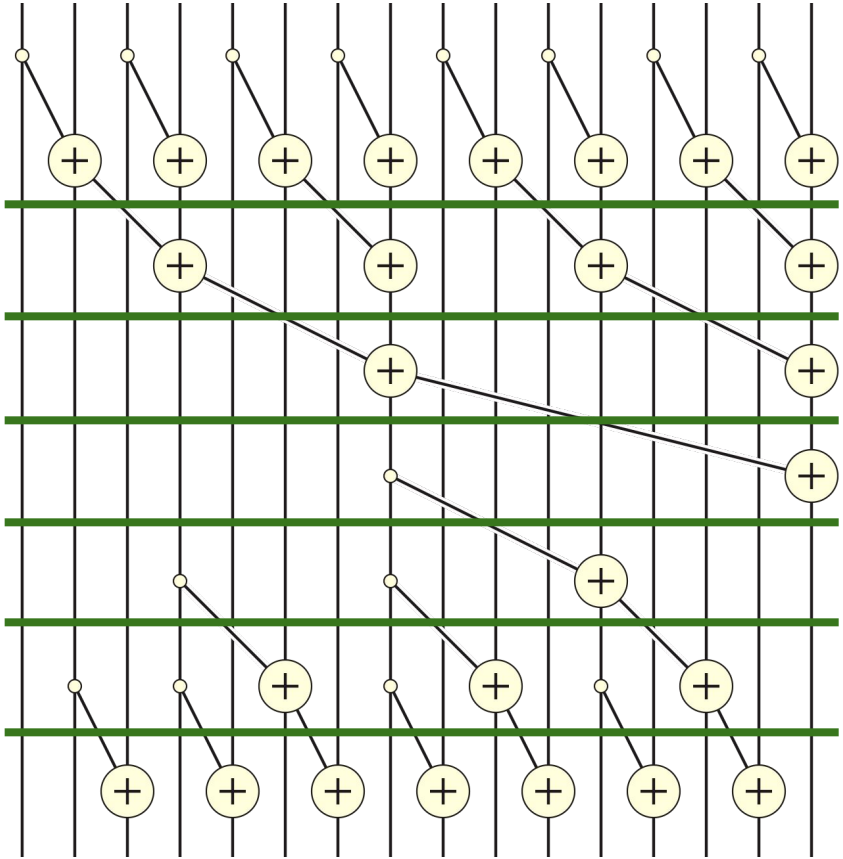
Runtime if there is **1** thread?

O(2n) **"work efficient"!**

Runtime if there are **n** threads?

Still O($\log_2 n$)

Again, this is what you'll implement in HW 2!



Miscellaneous Considerations

- *Manually building input files would suck… how can I make test files?*
  - Generate random input files with provided **`generate_input`** tool
  - Run **`prefix_sum`** providing the input file to actually test stuff, comparing to sequential
  - Refer to the README for more information (also for helpful debug info)

```
./generate_input      \
     -n <input_size> \
     -w <vec_width>  \
     -o <out_file>
```

→

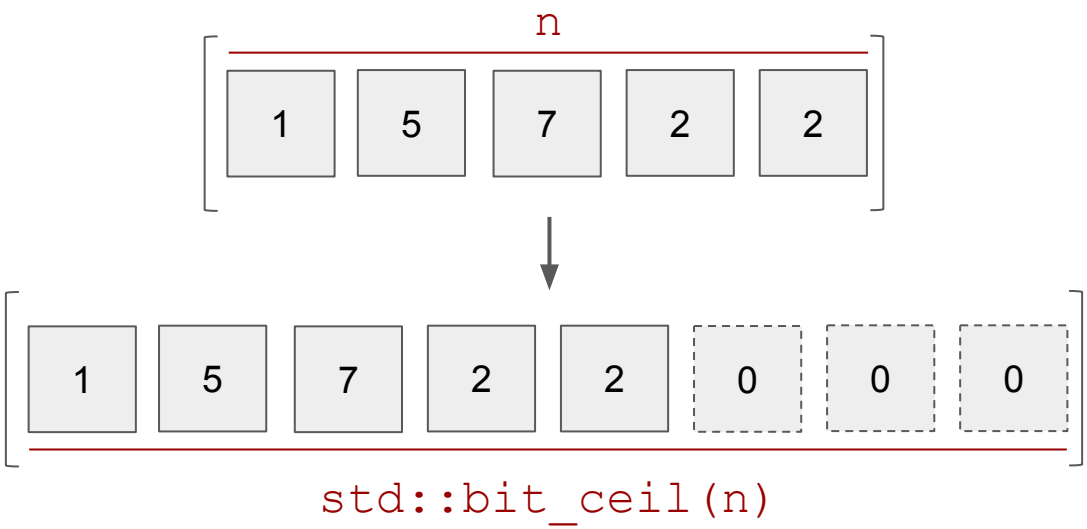
```
./prefix_sum          \
     -n <input_size> \
     -w <vec_width>  \
     -i <in_file>    \
     -o <out_file>   \
     -t <num_threads>
```

Miscellaneous Considerations

- *We've been looking at examples where n is a power of a two! What if thats not the case?*

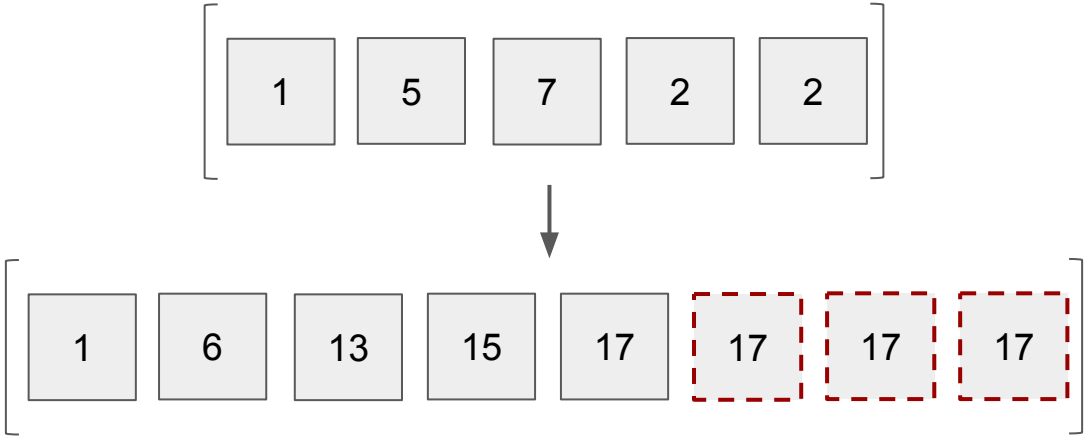
Miscellaneous Considerations

- *We've been looking at examples where n is a power of a two! What if thats not the case?*
  - We could just pad the input to a power of two?



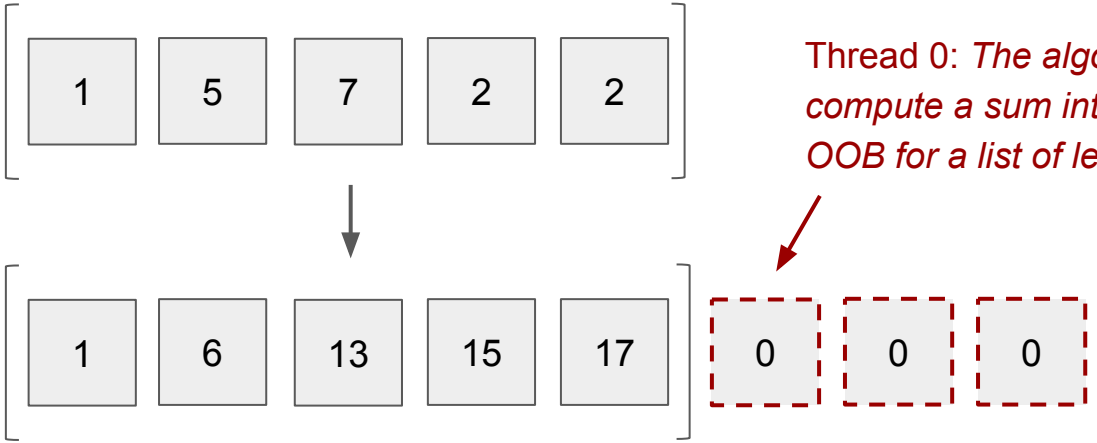
Miscellaneous Considerations

- *We've been looking at examples where n is a power of a two! What if thats not the case?*
  - We could just pad the input to a power of two?
    - But this wastes a lot of computing resources calculating results we don't need…



Miscellaneous Considerations

- *We've been looking at examples where n is a power of a two! What if thats not the case?*
  - We could just perform an in-bounds check before actually touching any memory, but otherwise *pretend* like n is a power of two.



Thread 0: *The algorithm is telling me to compute a sum into index 5, but index 5 is OOB for a list of length 5! So I'll ignore it…*

Miscellaneous Considerations

- *The HW questions require me to measure execution time, how do I do that?*
  - You are provided a Stopwatch in `/src/`!
  - Make graphs with your favorite graph-making tools (matplotlib, seaborn, etc…)

```
class Stopwatch {
public:
    void start();
    void stop();
    long long elapsed_microseconds() const;
    void print_elapsed() const;
};
```